PROJECT PROPOSAL

Enforcing Booked Slots on Research-Lab PCs with a Self-Hosted Headscale Server

A 15-machine pilot

Prepared by Zamiul Rashid, Akid Anis and Tanzim Ahmed

Supervisor Md. Golam Rabiul Alam · **Co-supervisor** Mollah Md. Saif · October 2026

Summary. Every thesis group already has booked slots on our lab PCs, but nothing enforces them. Groups work remotely through AnyDesk, and anyone with a PC's AnyDesk details can connect at any time. A student outside their slot can log in and disturb another group's running work. We propose a self-hosted system that enforces the slots. A Headscale server (an open-source control server for the Tailscale/WireGuard VPN) runs on a mini PC in the lab. Students sign in with their university email and a password on our own login server (Authelia). Every 12 hours a scheduler opens the PC to the incoming group over SSH and AnyDesk, and closes it to the outgoing group. It does this by changing the network policy, removing the outgoing group's devices, ending its sessions and changing the AnyDesk password. Outside its slot, a group cannot reach the PC at all. The pilot covers 15 Linux PCs. From the University we need only **one public IP address with three inbound ports open, and two DNS names**.

1 Background and Goals

Each thesis group is assigned two 12-hour slots per week on a lab PC and works remotely through AnyDesk. The slots exist only on paper. AnyDesk connects through AnyDesk's own servers, so anyone who knows a PC's ID and password can connect whenever they like. That person can stop another group's training run, change its files or take over its desktop, and nobody can tell afterwards who did it. Each group has at least five students, and students often use more than one device. Physical access stays under the lab's existing rules. The pilot aims to:

1. Enforce the slots: during a slot only that group can reach its PC remotely; outside it, nobody except the lab admins.
2. Run Headscale, a login server and a slot scheduler on one mini PC in the lab, reachable through one public IP address.
3. Enrol 15 Linux lab PCs, each registered once as a lab-owned machine.
4. Let students sign in with their university email and a password, with no Google or University IT involvement.
5. Switch access automatically at every slot change, for both SSH and AnyDesk, and record who connected when.
6. Keep only the current slot's devices on the server, and measure reliability, load and power use before any wider roll-out.

2 Options Considered: Why Headscale

Table 1 compares the main ways to give students remote access to lab PCs.

Table 1: Options for remote access.

Option	Control and data path	Fit for slot-based lab access
Current setup: AnyDesk only	AnyDesk's servers	No slot enforcement: anyone with a PC's ID and password can connect at any time, and there is no record of who did
SSH port-forwarding or a bastion host	Ports opened on campus; SSH exposed to the internet	Needs more than one IP and port from IT; open to brute-force attempts; slots enforced only on each PC
Plain WireGuard (hub)	Our server; all traffic through the hub	No NAT traversal; keys and firewall rules managed by hand for every device
Tailscale (hosted)	Tailscale Inc. (off campus); direct P2P	Can share a device into a group's own tailnet by email. The free plan covers 6 users; Standard costs $8 per user per month. The control plane and user data stay off campus
NetBird (self-hosted)	Ours; direct P2P	Capable, but several services to run (management, signal, relay, dashboard, identity provider) and its own clients
Headscale (chosen)	**Ours; direct P2P**	One binary plus SQLite, official Tailscale clients, free (BSD-3). The policy is one file we can regenerate and reload every 12 hours

3 Network Architecture

The design separates a **control plane** from a **data plane** (Figure 1). Headscale is an open-source reimplementation of Tailscale's coordination server that works with the official Tailscale clients. It registers devices, hands out tailnet addresses and keys, and pushes each device a peer list filtered by the access policy. It never carries user traffic. The data plane is WireGuard, running directly between the student's laptop and the lab PC.

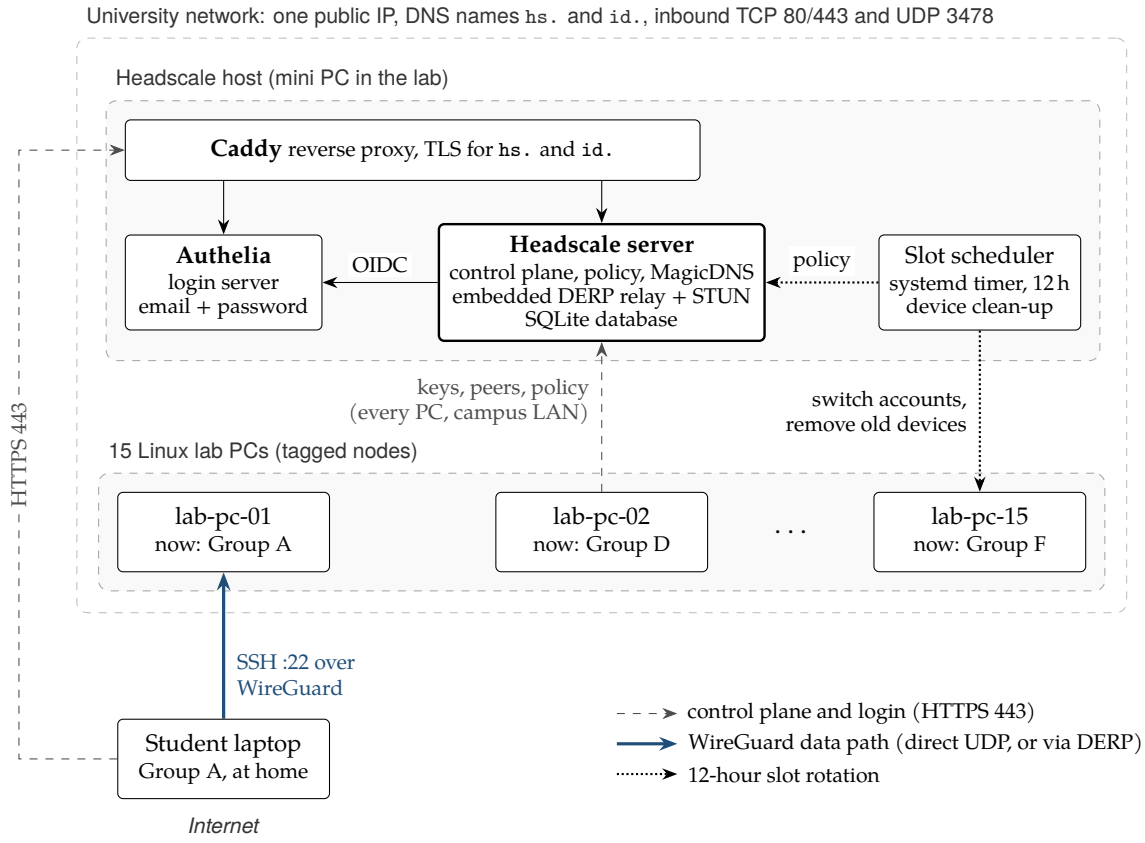


Figure 1: Pilot network architecture. Caddy is the only public entry point and routes `hs.[uni-domain]` to Headscale and `id.[uni-domain]` to Authelia. User traffic flows between laptop and lab PC, end-to-end encrypted with WireGuard.

How a connection is made.

1. Every device keeps an HTTPS (TCP 443) session to Headscale and receives only the peers the policy allows.
2. A new connection starts out relayed through a DERP server. Meanwhile, both ends learn their public addresses via STUN (UDP 3478) and attempt NAT traversal.
3. If a direct UDP path works, traffic moves to it (lowest latency). Under hard NAT, or where UDP is blocked, it stays on DERP over TCP 443, which is slower but works. The client keeps re-checking for a direct path.
4. Every path is WireGuard-encrypted end to end. The relay only ever sees ciphertext.

Our host runs Headscale's **embedded DERP relay**, so relayed traffic stays on campus. Tailscale's public DERP servers stay in the map as a backup. They can be removed to keep everything in-house, at the cost of a single point of failure.

One entry point. Authelia's login page needs its own HTTPS hostname next to Headscale's, but there is only one public IP and one port 443. Caddy terminates TLS for both names with automatic Let's Encrypt certificates and routes each name to its service. Headscale requires WebSocket support from a reverse proxy, which Caddy provides; Headscale's maintainers say they do not use proxies themselves, so the pilot tests this path early. STUN (UDP 3478) goes straight to Headscale. Lab PCs reach the server on its campus LAN address through a hosts-file entry, so they do not depend on the campus firewall looping traffic back to the public IP (hairpin NAT). For the same reason, the mini PC's own hosts file points `id.[uni-domain]` at itself, so Headscale reaches Authelia locally when it checks a login.

3.1 Machines, users and slots

Each **lab PC** is registered once with a single-use pre-authorised key that carries its tags (`tag:labpc` and its own `tag:pc-NN`). That makes it a *tagged node*, owned by the lab rather than by a person. Hosted Tailscale could share a device into another tailnet by email, but Headscale runs a single tailnet. Instead, every student is a user in our tailnet, identified by their university email. A **thesis group** is a list of emails in the access policy. "Sharing" a PC with a group is a policy grant that exists only during the group's slot. As with Tailscale's shared machines, the PC cannot start connections back to the students.

Each PC has 14 slots per week, **08:00–20:00** and **20:00–08:00** every day. Each group gets two slots on different days (Table 2). At each boundary, a systemd timer on the host:

1. writes the policy for the new slot from the schedule and roster files and reloads Headscale (`systemctl reload headscale`). All 15 PCs change in one batched update;
2. removes the outgoing groups' devices from Headscale (Section 3.3);
3. logs in to each PC over SSH as the scheduler node, ends the outgoing group's sessions and processes, and moves the `slot-active` Linux group from the outgoing group's account to the incoming one;
4. on any error, loads an admins-only policy and raises an alert, so access fails closed. Having no policy at all would allow everything.

Table 2: Example weekly schedule for one PC: six groups (A–F) and one maintenance slot.

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
08:00–20:00	A	C	E	A	C	E	*maint.*
20:00–08:00	B	D	F	B	D	F	*spare*

```
"groups": { "group:thesis-a": ["s2101@[uni-domain]", "s2102@[uni-domain]", ...] },
"grants": [
  { "src": ["group:admins", "tag:scheduler"], "dst": ["tag:labpc"], "ip": ["tcp:22"] },
  // generated for the current slot (Mon 08:00-20:00): pc-01 shared with Group A
  { "src": ["group:thesis-a"], "dst": ["tag:pc-01"], "ip": ["tcp:22"] } ]
```

Lab PCs. Each PC runs Ubuntu LTS with the Tailscale client and plain OpenSSH. Login is key-only, `AllowGroups lab-admins slot-active` restricts who may log in, and the firewall accepts SSH only on `tailscale0`. Each group has one Linux account holding its members' keys. Home directories persist between slots, but running jobs stop at slot end, so groups must checkpoint.

3.2 Student login with Authelia

We run our own login server, **Authelia**, on the mini PC. Headscale does not check passwords itself: it hands student sign-in to Authelia over OIDC. Authelia's documentation includes a Headscale setup, tested with Headscale 0.29.1.

What the student does. On the first day, the admin gives each student their username and a starting password in private. At the start of each slot the student:

1. runs `tailscale login --login-server` `https://hs.[uni-domain]`, or presses "Log in" in the Tailscale app. A browser tab opens on our login page at `id.[uni-domain]`;
2. enters their university email and password. The tab then reports success;
3. connects with `ssh grp-a@lab-pc-01`, or with AnyDesk using the slot's password. This works only during their group's slot.

Behind the scenes, Authelia tells Headscale "this is s2101@[uni-domain], member of `lab-pilot`". Headscale registers the laptop under that email, and the slot policy decides which PC, if any, it can reach. *Signing in proves who you are; the schedule decides what you can reach.* Students can keep a personal Tailscale account alongside ours, because client 1.34 and later supports several accounts.

- **Accounts.** The admin adds each student to Authelia's user file with their university email and the group `lab-pilot`. Headscale admits only that group (`allowed_groups`). Passwords are stored only as Argon2id hashes, never in plain text.
- **Password guessing.** Authelia's brute-force protection bans an account or IP address after repeated failed logins (by default, 3 failures within 2 minutes trigger a 5-minute ban). We enable both modes.
- **Password resets.** The admin resets forgotten passwords, so no email server is needed. Self-service reset by email can be added later with an SMTP account. Two-factor codes (TOTP) can also be switched on later by changing the client's policy from `one_factor` to `two_factor`.

```
users:                                  # Authelia user file: one entry per student
  s2101:
    displayname: "Student Name"
    email: "s2101@[uni-domain]"
    password: "$argon2id$v=19$m=65536,t=3,p=4$..."    # hash only, never the password
    groups: ["lab-pilot"]               # Headscale: allowed_groups: ["lab-pilot"]
```

Why not Google or University SSO. Google sign-in depends on the University's Google admin settings, which can block apps like ours, and a personal Google project is limited to 100 test users. Our own login server needs nothing from IT. Authelia's Headscale guide is community-maintained, so the pilot first checks that Headscale receives each student's email and groups correctly.

3.3 Devices and node budget

A *node* is a device, not a user: a student with a laptop and a phone is two nodes. Each group has at least five students, and devices accumulate fast if they stay registered (Table 3):

Table 3: Nodes on the server (15 lab PCs and 1 scheduler included).

Situation	1 device / student	2 devices / student
One slot, all 15 PCs in use, 5 students per group	91	166
Devices kept all week (6 groups per PC, 90 groups)	466	916

Headscale's FAQ says it "can likely handle 100s of devices (maybe more)" when the network changes rarely, because every change rebuilds the network map for every node. We therefore keep **only the current slot's devices** on the server:

- At slot end the scheduler deletes the outgoing groups' devices. Students sign in again with their password at the start of their next slot, which takes a few seconds.
- Every five minutes, a clean-up job removes devices of students who are not in an active slot, and any device beyond **two per student**.

The server then holds at most about **166 nodes** (91 with one device per student) instead of 466–916. Student *accounts* grow with the roster, but accounts are only database rows and do not affect performance. The main load is the slot changeover twice a day, when up to about 150 devices leave and rejoin. The pilot measures how long that takes.

3.4 Remote desktop in the slot (AnyDesk)

AnyDesk is how groups work today, and it is where the slots are bypassed. It stays available, but only inside the slot. There are two caveats.

1. AnyDesk normally connects through *AnyDesk's own servers*, not our tailnet, so the Headscale policy cannot block it. Instead, the AnyDesk password becomes the slot key. At every slot change the scheduler sets a new unattended-access password (`anydesk --set-password`, from AnyDesk's Linux CLI), gives it only to the incoming group, and restarts AnyDesk to drop the outgoing session. The old password stops working the moment the slot ends. The PC's desktop login also switches to the incoming group's account.
2. AnyDesk's free licence is for private use only. Lab use counts as professional and needs a paid licence.

If no licence is available, **RustDesk** (open source) can connect directly to the PC's tailnet address. The same slot policy then controls it, just like SSH.

4 Technical Specifications

The whole system runs on one small, low-power machine in the lab (Table 4).

Table 4: Headscale host: hardware and power.

Machine	Intel N100-class mini PC (4 cores), 8 GB RAM, 128–256 GB SSD, 1 GbE wired
Software	Ubuntu Server 24.04 LTS; Headscale (latest stable release, pinned) with SQLite and embedded DERP; Authelia; Caddy; slot scheduler (systemd timer and script)
Power draw	N100 mini PCs measure about 6–12 W idle and 22–29 W at full load; about 7 kWh per month at a 10 W average. We will measure our unit
Backup power and data	650 VA UPS. A nightly copy of the Headscale and Authelia databases, user file, keys and configuration goes to a second disk

Section 5 lists the ports exposed to the internet. On the lab PCs, SSH (TCP 22) is accepted only on the `tailscale0` interface.

5 What We Need from the University

1. **One static public IPv4 address**, routed or forwarded to the mini PC.
2. **Two DNS records** for that address: `hs.[uni-domain]` (Headscale) and `id.[uni-domain]` (login server).
3. **Three inbound ports open from the internet** to that address, and no others:

Table 5: Ports exposed to the internet (inbound to the public IP only).

Port	Received by	Purpose	Status
TCP 443	Caddy	Student login (Authelia), Tailscale control traffic to Headscale, and the DERP relay	Required
UDP 3478	Headscale	STUN, so devices can find their public address and connect directly	Required
TCP 80	Caddy	Let's Encrypt certificate checks; redirects browsers to HTTPS	Recommended*

*Caddy can also validate certificates over TCP 443, so port 80 can stay closed if the University prefers.

Nothing else is exposed. The lab PCs, SSH (port 22) and the Headscale and Authelia services themselves are not reachable from the internet. All student traffic reaches a PC only through Caddy (login and control) or WireGuard (SSH), and only during the group's slot.

Outbound traffic from the lab network, which campus networks normally already allow: UDP 41641, UDP 3478 and TCP 443 to any address, from the lab PCs and the mini PC. If outbound UDP is blocked, connections still work through our relay on TCP 443, only slower. AnyDesk keeps using the outbound connections it uses today.

The project team provides everything else: hardware, software, certificates, student accounts, PC setup and operation.

6 Risks

Risk	Mitigation
Scheduler fails at a slot boundary	Falls back to an admins-only policy (no student access) and alerts the admins
Slot changeover is slow (about 150 devices leave and rejoin)	Devices are removed in one batch; changeover time is measured in the pilot; device limit can drop to one per student
Authelia is down	Already-connected devices keep working; only new sign-ins wait. Authelia is backed up nightly
A password is guessed or shared	Brute-force bans; the admin resets the password; two-factor codes can be switched on; a stolen login still reaches only that group's PC during its slot
AnyDesk bypasses the tailnet	The password is rotated and the session ended at every boundary; licensed use only, or RustDesk over the tailnet
Headscale host fails	UPS and nightly backups; existing tunnels survive short outages